
ISL

Improvement Software Loop

An autonomous control plane that takes any codebase you point it at, works out what is worth changing, makes the change in isolation, and proves the result before it is allowed to land.

Executive summary

Software decays faster than teams can maintain it. The backlog of work that everyone agrees should happen — the missing test, the function nobody dares refactor, the dependency with a published CVE, the accessibility gap — loses every week to work with a deadline attached. ISL exists to do that work continuously, and to prove it did it correctly.

12

deterministic vetoes before any commit

13

specialist agents, routed by learned competence

10

pipeline phases per iteration

0

changes to your main branch or working tree

What it is

ISL is not a code generator with a chat box. It is a loop with a memory. It reads a project and its documentation, decides what to work on, sends the task to the specialist with the best record on that kind of work, applies the change inside a disposable sandbox, and then submits the result to a stack of checks that owe nothing to the model that produced it. A change that fails any of them is discarded, the target is charged a failure, and the next plan is different because of it.

Why the guardrails are the product

Any capable model can write a plausible patch. The difficult part — and the part that decides whether an autonomous loop is an asset or a liability — is refusing the plausible patch that is wrong. ISL's checks are deterministic: they do not ask a model for an opinion, they can each be traced to the exact line that triggered them, and no score overrides them. This document is explicit about the failures that produced each one, because a guardrail whose origin is a real incident is easier to trust than one derived from principle.

What it is for

Point ISL at a folder. Nothing needs to be instrumented, ported or rewritten first. It catalogues what is there, builds a grounded picture of the project from its own documents, asks a short list of questions only where the documents genuinely do not answer, and starts work.

Continuous maintenance

Complexity, dead code, missing tests, stale documentation and structural debt — worked through steadily rather than in an annual clean-up that never arrives.

Security & compliance

Secrets and weakened controls caught in the diff; dependency CVEs found and safe upgrades computed; best-practice rules audited across every language in the repo.

Reliability

The app is booted on every iteration. A change that compiles, passes tests and stops the product from starting does not land.

Evidence

Every commit records what produced it, what checked it, and what each check found — the audit trail a regulated team has to produce anyway.

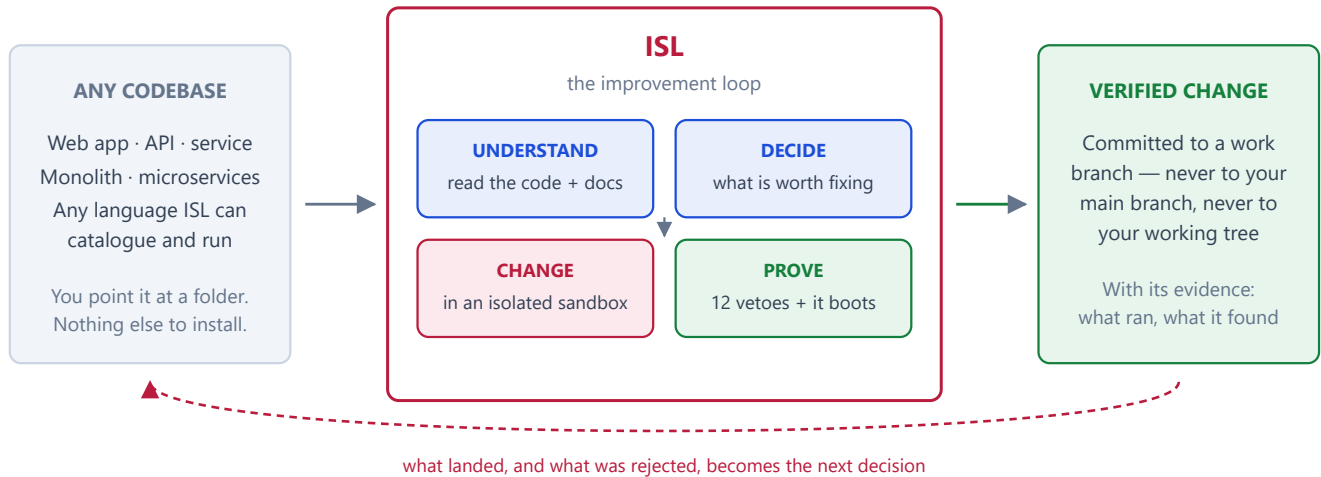


Figure 1. Any codebase in, a verified change out — and the outcome of that change becomes an input to the next decision. Nothing is written outside a sandbox, and nothing reaches your main branch without a human or an explicit policy allowing it.

Where a human stays in the loop

Autonomy is a dial, not a switch. Low-risk work by an agent with a proven record can land on the work branch on its own; anything risky, or anything from an agent still earning trust, waits in an approval inbox. Promotion to your main branch is a separate, explicit act that is off by default. A rejection is not discarded — it becomes a durable lesson and costs the agent its standing on that kind of work.

How it works

One iteration is ten phases, run inside a git worktree cut from the work branch. The first four decide and produce a change; the last six try to find something wrong with it.

ONE ITERATION

Every run is a sandbox cut from the work branch. Nothing outside it is touched.

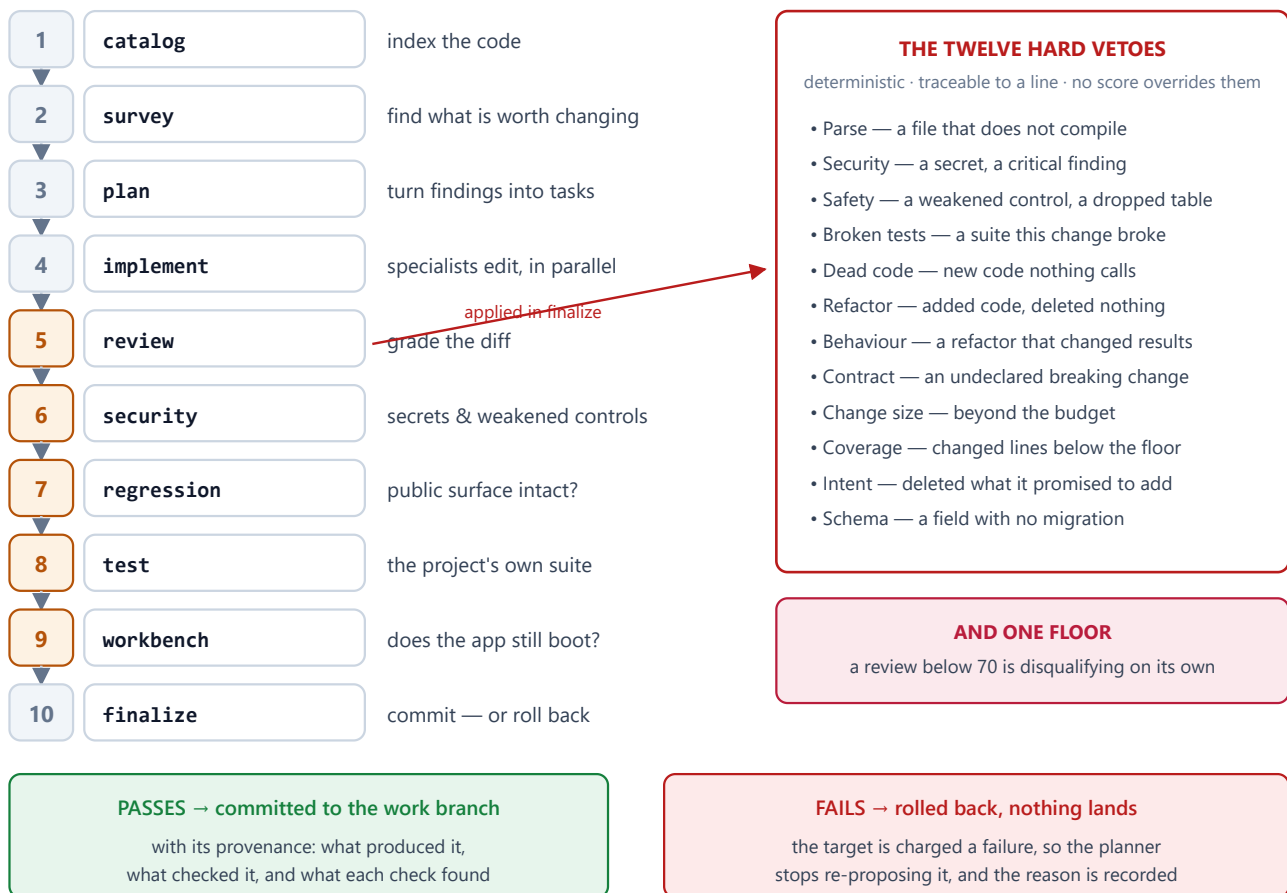


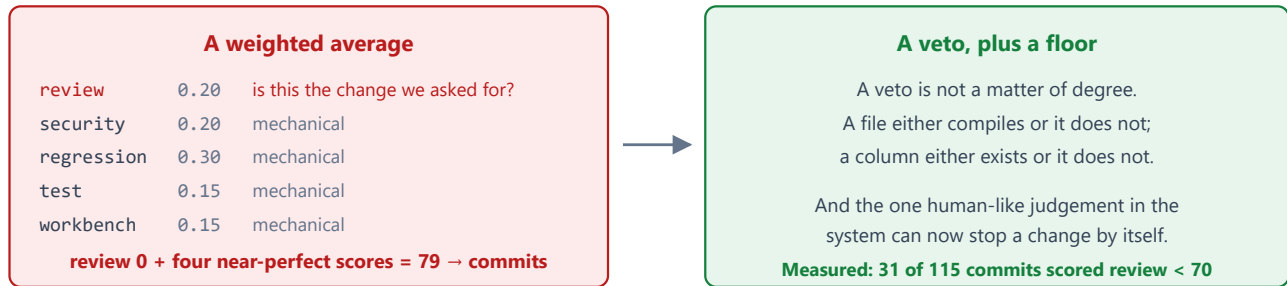
Figure 2. The iteration pipeline. Phases 5–9 are the graders; the vetoes are applied together in `finalize`, so a change is judged as a whole rather than abandoned halfway. A rollback costs one iteration and leaves no trace outside the sandbox.

Why a score was not enough

ISL originally decided by weighted average against a threshold. That failed in a way worth documenting, because the failure is arithmetic rather than bad luck: the grader that asks *“is this the change we asked for?”* carries a weight of 0.2, while four checks measuring mechanical properties of the result carry 0.8 between them. A review of zero alongside four near-perfect mechanical scores totals 79 — comfortably above a rollback threshold of 60.

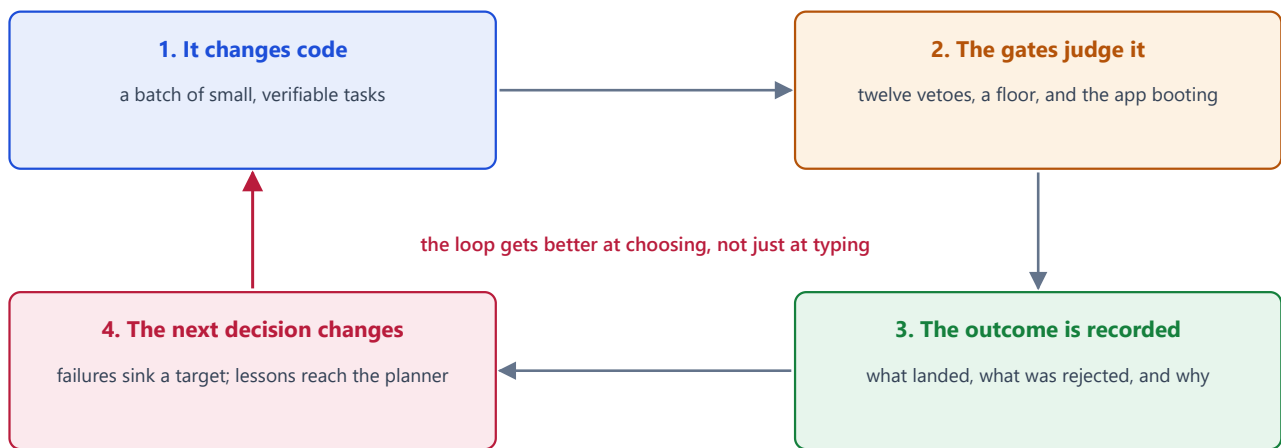
Four changes that reached the target application in production were traced back to exactly this. In every one, the reviewer had objected and been outvoted.

THE ARITHMETIC THAT LET FOUR BAD CHANGES THROUGH



Evidence, not preference: the committed review scores fall in two clusters, 0–55 and 75–100, with nothing between them. The floor was placed in that gap.

Figure 3. A weight cannot stop anything on its own. The fix was not to re-tune the weights but to add vetoes — conditions that are not matters of degree — and a floor under the one judgement in the system.



This is what separates ISL from a code generator: the result of a change is an input to the next one.

Figure 4. The loop improves at choosing, not only at writing. A failed target sinks in the ranking, a rejected change becomes a recorded pitfall, and the agent that produced it loses standing on that kind of work.

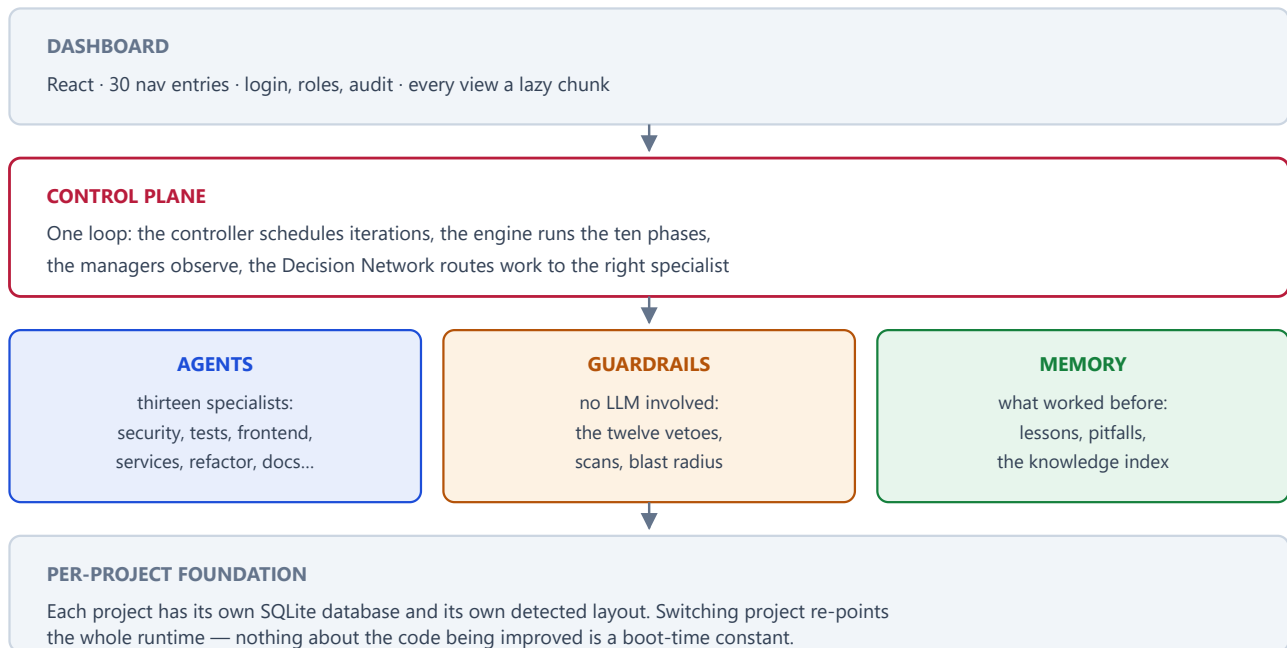


Figure 5. The layers. Nothing about the project being improved is fixed at boot: configuration values are live bindings and the database handle is swappable, so switching project re-points the entire runtime without a restart.

WHAT ISL MAY AND MAY NOT TOUCH

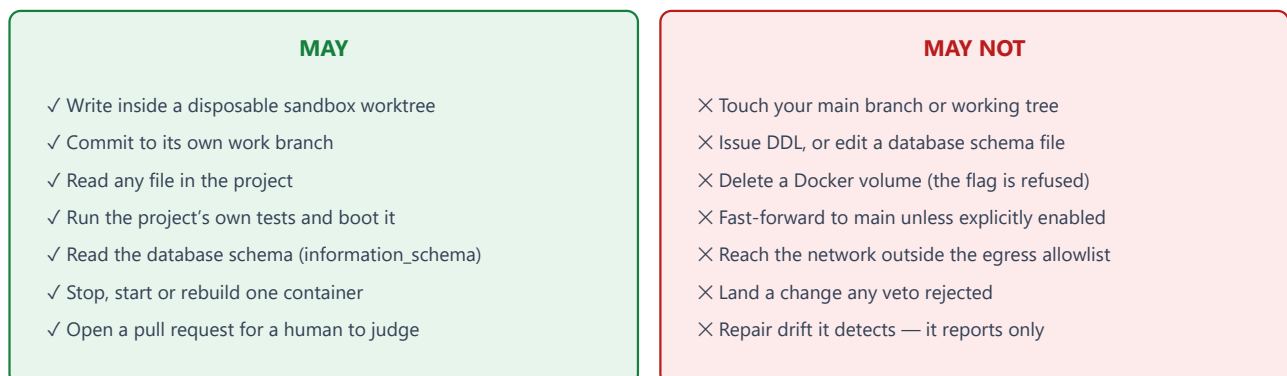


Figure 6. The boundary is enforced in code, not documented as a convention. The flag that would delete a database volume is refused by the runtime, and a test reads the source to make sure no future command reintroduces it.

Contents of the technical reference

Everything that follows is the complete text of `ISL.md`, the maintained technical reference, reproduced without omission. Its own table of contents is retained; this listing is generated from the document as printed.

ISL — Improvement Software Loop

Table of contents

Part I — What it is

1. The big picture
2. The self-improvement flywheel
3. Design principles

Part II — How it is built

4. Architecture
5. Technology stack
6. The multi-project foundation
7. The iteration pipeline
8. The sandbox
9. Iteration lifecycle & restarts

Part III — The intelligence

10. The 13 agents
11. The 13 managers
12. The Decision Network
13. Shared Memory
14. Context — the project's ground truth
15. The knowledge index (hybrid RAG)
16. Models & LLM providers

Part IV — The guardrails

17. Deterministic gates
18. Code intelligence
19. Coverage
20. The public-contract gate
21. Human review & trust

22. Self-correction & reliability

Part V — Enterprise controls

23. The egress firewall
24. Tamper-evident audit & evidence packs
25. RBAC & segregation of duties
26. Policy-as-code
27. Cost & capacity governance
28. The self-evaluation harness
29. Governance, compliance & best practices

Part VI — Steering & operations

30. Scope — who decides what gets improved
31. Autonomy & scheduling
32. Deployment, promotion & cloud
33. Online research

Part VII — The dashboard

34. Navigation & shell
35. The views
36. The components worth knowing

Part VIII — Reference

37. Running ISL
38. Configuration
39. API reference
40. Data model
41. Events
42. Source map
43. Troubleshooting
44. Glossary

ISL — Improvement Software Loop

ISL is an autonomous, multi-project control plane that continuously improves a codebase. It catalogues a project, decides what to improve, routes each change to the specialist most likely to land it, writes the code in a throwaway sandbox, grades it against deterministic gates, commits only what passes — and does it again, forever, **learning from every outcome**.

Model-agnostic and language-agnostic. It ships pointed at a first project (RentAll) but manages **any** application: point it at a folder and it analyses the real layout, detects the languages, and works on them.

Scale, as of 2026-07-26: 114 backend modules (~24,600 lines), 36 dashboard views and 26 components (~12,600 lines), 201 HTTP endpoints, 43 database tables, 13 improvement agents, 13 managers, 53 language toolchain adapters.

Table of contents

Part I — What it is

1. The big picture
2. The self-improvement flywheel
3. Design principles

Part II — How it is built

1. Architecture
2. Technology stack
3. The multi-project foundation
4. The iteration pipeline
5. The sandbox
6. Iteration lifecycle & restarts

Part III — The intelligence

1. The 13 agents
2. The 13 managers
3. The Decision Network
4. Shared Memory
5. Context — the project's ground truth
6. The knowledge index (hybrid RAG)
7. Models & LLM providers

Part IV — The guardrails

1. Deterministic gates
2. Code intelligence
3. Coverage
4. The public-contract gate

5. [Human review & trust](#)
6. [Self-correction & reliability](#)

Part V — Enterprise controls

1. [The egress firewall](#)
2. [Tamper-evident audit & evidence packs](#)
3. [RBAC & segregation of duties](#)
4. [Policy-as-code](#)
5. [Cost & capacity governance](#)
6. [The self-evaluation harness](#)
7. [Governance, compliance & best practices](#)

Part VI — Steering & operations

1. [Scope — who decides what gets improved](#)
2. [Autonomy & scheduling](#)
3. [Deployment, promotion & cloud](#)
4. [Online research](#)

Part VII — The dashboard

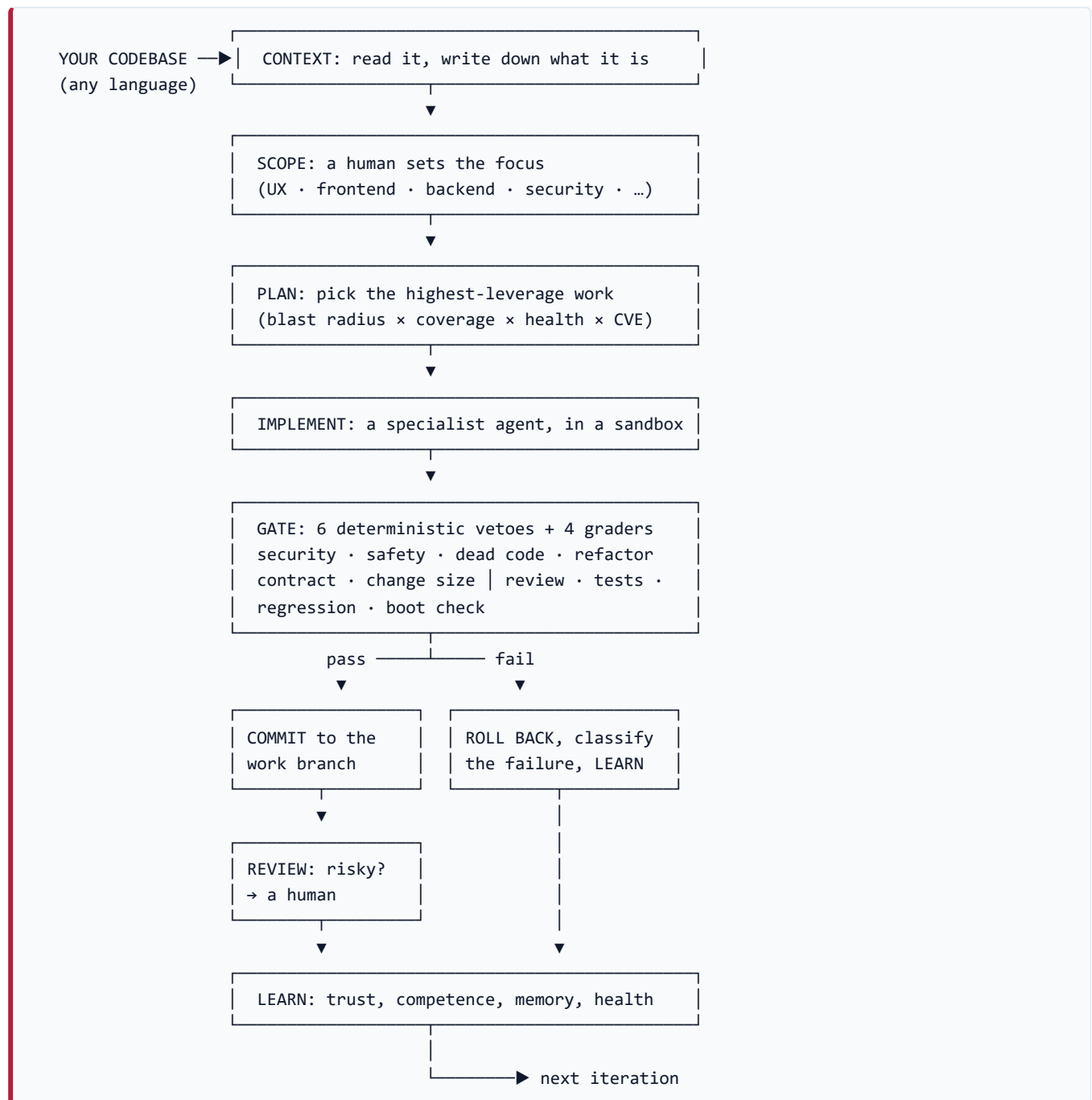
1. [Navigation & shell](#)
2. [The views](#)
3. [The components worth knowing](#)

Part VIII — Reference

1. [Running ISL](#)
 2. [Configuration](#)
 3. [API reference](#)
 4. [Data model](#)
 5. [Events](#)
 6. [Source map](#)
 7. [Troubleshooting](#)
 8. [Glossary](#)
-

Part I — What it is

1. The big picture



Your `main` branch is never touched. Everything lands on a work branch (`agents/auto-improve` by default), and promotion to `main` is a separate, gated act.

2. The self-improvement flywheel

Each turn of the loop produces evidence that makes the next turn better:

OUTCOME	WHAT ISL LEARNS
A change lands	The agent's trust rises; its competence in that area rises; the approach is indexed as a "proven approach" for similar future work
A gate vetoes a change	The failure is classified, a scoped pitfall is written to shared memory, and the agent's trust falls
A human rejects a change	The strongest signal available — an agent-scoped pitfall memory, and a trust demotion that no machine-gate pass can override
A regression appears	Auto-bisect finds the culprit commit; the blame goes to the right change rather than the current one
Health drops	Autonomy narrows: auto-land latitude is revoked for everyone until it recovers

3. Design principles

These are not aspirations; they are decisions visible in the code and enforced by tests.

Deterministic beats clever. Every veto is a pure function of a diff. A model's opinion can lower a score but can never override a gate. A gate that depends on a model is a gate that fails when the model has a bad day.

Fail closed. The egress firewall's default refuses cloud calls. Budgets ship off. The policy engine ships empty. An install that upgrades into a new control must not start behaving differently because nobody configured it yet.

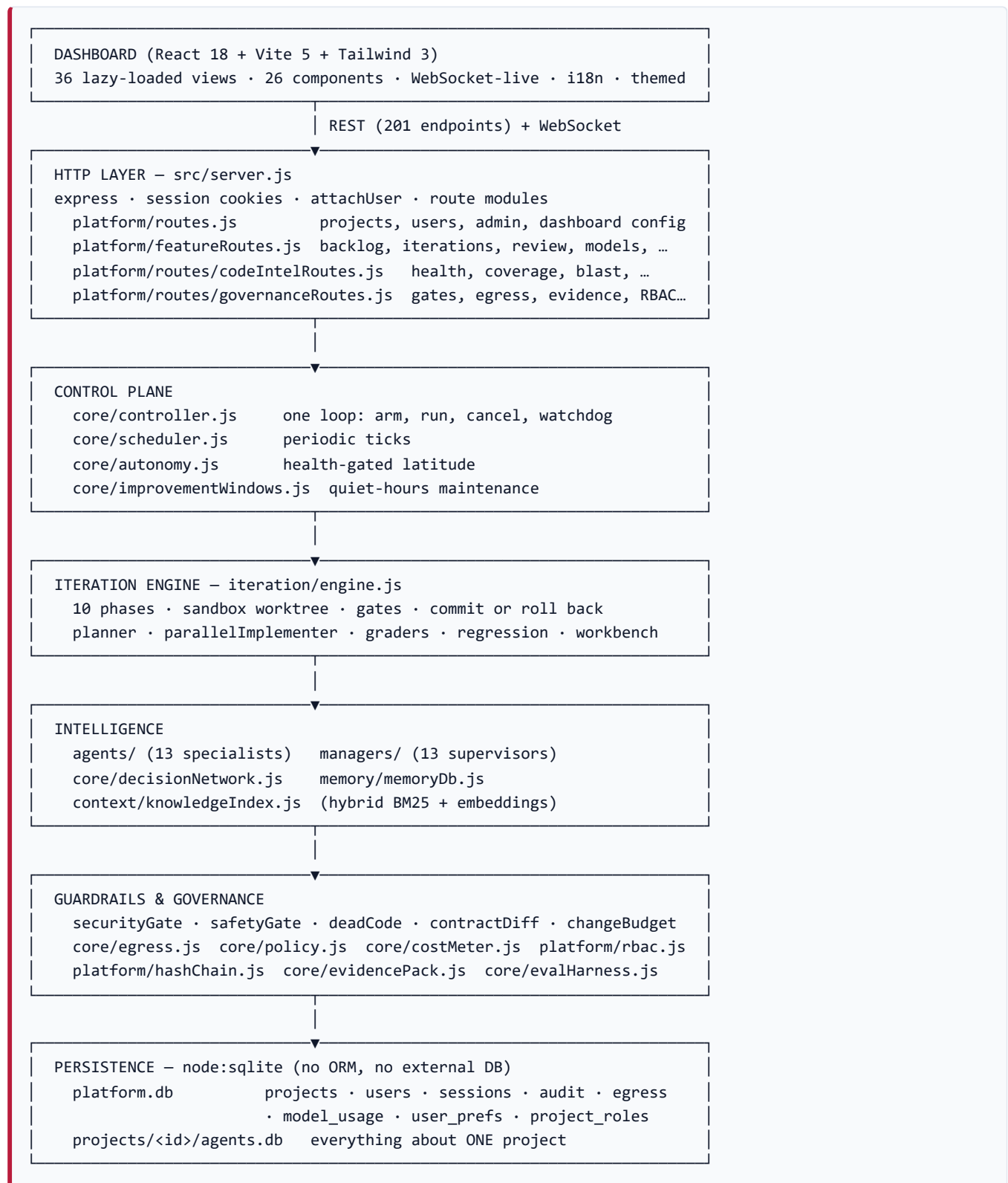
Never report an absence as a fact. Coverage that was never measured shows "not measured", never 0%. A model with no configured price is "unpriced", never free. An agreement rate from three samples is reported as underpowered rather than as a percentage. This rule is why several features carry visible caveats: the caveats are the feature.

Layer, don't replace. Organisational policy can only *tighten* what the built-in gates decided. Replacing battle-tested veto logic with a config file means a gap in the config becomes a gap in the guardrails — and that failure is silent.

The loop never stalls on a human. Review is advisory on the commit: risky changes still land on the work branch and wait for a human blessing before they can be promoted. A queue that blocks the loop is a queue that gets bypassed.

Part II — How it is built

4. Architecture



The single most important architectural decision is that nothing about the code being improved is a boot-time constant. `config.js` exports **live bindings** (`export let REPO_ROOT, ...`) that `setActiveProjectConfig()` reassigns on a project switch. Every consumer reads them at call time and therefore follows the active project

for free.

The rule this creates: never capture a live binding into a module-level `const`. Doing so freezes that module to whichever project happened to be active at import time. Every module in `src/` follows this, and it is the first thing to check when a feature "works on project A but not project B".

Process-wide caches keyed to "the code" must die with a project switch. `activeProject.js` invalidates the blast-radius graph, the coverage memo and the egress tripwire fingerprints in one place, so the leak is structurally impossible rather than a bug to remember.

5. Technology stack

LAYER	CHOICE	WHY
Runtime	Node 22+ (ESM)	<code>node:sqlite</code> built in — no native module to compile
HTTP	express 5	Thin; the routes are the interesting part
Realtime	<code>ws</code>	One socket, one event stream, no polling
Database	<code>node:sqlite</code> (<code>DatabaseSync</code>)	Zero-install, synchronous, transactional; a control plane's data is small and its reads are hot
Local model	Ollama (<code>qwen3.6:latest</code>)	Runs on the operator's machine; nothing leaves by default
Cloud models	Any OpenAI-compatible endpoint	One adapter covers OpenAI, OpenRouter, Groq, Together, vLLM, Ollama's <code>/v1</code>
Embeddings	<code>nomic-embed-text</code> (768-dim)	Local, good enough for code retrieval
Frontend	React 18 + Vite 5 + Tailwind 3	Route-level code splitting; hash routing; no router dependency
Frontend tests	vitest + Testing Library + axe	72 tests, including automated accessibility checks
Dependencies	7 runtime packages	<code>cors express ws diff dotenv mammoth pdf-parse</code> . Everything else is standard library.

Deliberate non-dependencies. No ORM (SQL is the query language). No syntax-highlighting library (a 60-line tokenizer covers a diff). No virtualisation library (fixed-height windowing is a dozen lines). No glob library (`src/glob.js` is 50 lines). Each would have been a large addition to a bundle whose entire value proposition is being fast to operate.

6. The multi-project foundation

Two database tiers:

- `platform.db` — one, shared: the project registry, users, sessions, the audit chain, the egress ledger, model usage, per-user preferences, per-project role grants.
- `.data/projects/<id>/agents.db` — one per project: agents, iterations, phases, tasks, backlog, proposals, context, memory, health snapshots, the review queue, knowledge embeddings.

`db.js` exports a **Proxy** over a swappable handle. `openProjectDb(path)` repoints it; every consumer keeps its import and follows the switch. This is what lets ISL manage several codebases from one process without a restart.

Switching a project's **source folder** invalidates the analysis: the backlog is reset, the function catalogue is dropped, and every derived context is cleared, because a plan built against a different codebase is worse than no plan.

7. The iteration pipeline

Ten phases, in order. Each is timed, scored, recorded in `phases`, and streamed as an event.

#	PHASE	WHAT IT DOES
1	catalog	Index the code: functions, files, hotspots, complexity
2	survey	Every N runs, re-read the repo and refresh the backlog with grounded items
3	plan	Choose this batch's work; route each task to a specialist via the Decision Network; attach the blast radius, proven approaches, and memory
4	implement	Run agents in parallel in the sandbox, each with a bounded tool loop
5	review	LLM code review — correctness, wiring, dead code
6	security	LLM security review, on top of the deterministic gate
7	regression	Golden-surface comparison: did anything that used to work stop working?
8	test	Run the project's own suite via the detected adapter
9	workbench	Boot the application. The check every unit test in the world sails past
10	finalize	Gates, score, commit or roll back, learn

Phase 9 earns its keep. A change can pass review, security, regression and the full test suite and still leave an app that does not start. `workbench.js` boots it in the sandbox and refuses the change if it does not come up.

The twelve hard vetoes

Applied in `finalize`, before anything is committed. Each is deterministic, each can be traced to the exact line that triggered it, and no score overrides them.

VETO	REFUSES
Parse	A changed file that does not compile
Security	A secret in the diff; a critical SAST finding
Safety	A change that <i>weakens</i> security — TLS verification disabled, an auth check deleted, a table truncated
Broken tests	A suite that PASSED at the base commit and fails after the change
Dead code	New code nothing calls; a new file nothing imports
Refactor	A "structural change" that added code and deleted nothing (it duplicated instead of replacing)
Behaviour	A refactor whose suite results differ from the base — including <i>fewer tests collected</i>

VETO	REFUSES
Contract	An undeclared breaking change to the public surface (see §20)
Change size	A diff beyond the configured budget
Coverage	Changed lines below the floor, when the coverage gate is in <code>enforce</code> mode
Intent (2026-08)	A task that committed to adding or fixing <code>X</code> , producing a diff where <code>X</code> is deleted and never re-added (§17.1)
Schema (2026-08)	A Prisma model field added with no migration in the same change — a column that will not exist (§17.2)

Alongside them, one **floor** rather than a veto on the diff: a review score below `kpi.review_floor` (default 70) is disqualifying on its own. See §17.1 for why the weighted average made this necessary, and the score distribution that put the threshold where it is.

Why a score could never have done this

Three of these were scores before they were vetoes, and the arithmetic is the reason that failed. The weighted total is `review 0.2 · security 0.2 · regression 0.3 · test 0.15 · workbench 0.15` against a rollback threshold of 60. So:

- a change with **every test red** and everything else near-perfect totals **83** — it commits;
- a change with a **file that does not compile** (review returns 0) totals **~80** — it commits.

Measured on real runs before the fix: six of the last twenty-five committed with `test = 0`, and two committed with a syntax error in the diff. No weight assignment fixes this without making one gate the only thing that matters — a failure of that kind is not a matter of degree, so it is a veto.

The same arithmetic caught up with the reviewer (2026-08). Review is the only grader that asks whether the change is the one that was *requested*; the other four measure properties of the result. At a weight of 0.2 its verdict is arithmetically incapable of stopping anything: a review of **0** beside four near-perfect mechanical scores totals **79**.

Four changes that broke the target application in production were traced back this way — a schema field with no migration (review 35, total 63), two files with syntax errors that crash-looped their containers (review 0, totals 79 and 80), and a deleted feature (review 55, total 90). In every case the reviewer objected and was overruled by the average. Across 115 commits, **31 carried a review below 70**. The committed scores fall in two clusters, `0-55` and `75-100`, with nothing between — so the floor was placed in that gap rather than chosen.

Attribution: a failure you did not cause is not your failure

The test grader re-runs a failing suite at the **base commit**. A suite that was already red is recorded as `inherited`, reported prominently, fed to the auto-bisect — and **excluded from the score**. Only suites the change actually broke count against it.

Before this, a pre-existing failure scored the change 0, which did two kinds of harm: a good change graded 83 instead of 98, and — far worse — `test = 0` became the normal state, so a change that genuinely broke the suite looked exactly like one that had not. The veto above depends on this distinction being made honestly.

Portability: the agent tools do not assume a platform

Two things that look like details decide whether the fleet can edit anything at all on a given machine, so they are stated here rather than left in the code:

- **Line endings are not part of a match.** `edit_file` matches ``` against ```, because a model quoting a CRLF file back will emit ```. Requiring a byte-exact match made **103 tasks** end with "the implementer finished without editing any file" on Windows, while the identical code worked on Linux. The **write** is not tolerant: only the replacement's endings are converted to the file's convention, so untouched bytes stay byte-identical and a three-line edit stays a three-line diff.
- **A new file follows the repository, not the host.** The convention is sniffed from `.gitattributes` and real source files — never from `process.platform`, because a Windows machine can hold an LF repository and a Linux CI can check out CRLF.

What the fleet is allowed to learn

Self-regulation is only real if what the fleet learns can change what it does. `memory/lessons.js` holds the single rule: **a memory earns a place in a prompt only if it would change what the agent does next.**

- Failures that describe the **machine** (a dropped connection, a full disk) or the **operator** (a cancellation, a restart), a base that moved under a change, and failures our own classifier does not recognise are recorded, clustered and shown — but never taught.
- Everything else maps to a **curated** lesson that names the tool and the step that was skipped. Generated summaries of error messages are exactly what this replaced: the fleet spent months being told "Recurring error: fetch failed" at the top of every prompt.
- `recall` ranks **pinned** → **scope-specific** → **recurrence** → **recency**. Ranking by recurrence alone let the most frequent infrastructure problem outrank a lesson written once about the agent's own work.

8. The sandbox

Every iteration runs in a **detached git worktree** cut from the tip of the work branch. The operator's checkout is never written to.

`node_modules` is **junctioned** from the real checkout rather than reinstalled — ~1s instead of ~60s. This is a deliberate speed/isolation trade with one sharp edge, documented in `sandbox/worktree.js`: anything that *installs* or *mutates* dependencies must not use `createSandbox`. CVE remediation therefore creates a **raw** worktree and runs `npm audit fix --package-lock-only`, which installs nothing — verified by checking that the real `package-lock.json` md5 is identical before and after.

Known gap, stated plainly: the junction means a test with a side effect on `node_modules` reaches the developer's machine. The fix is a containerised verification sandbox (`ISL_IMPROVE.MD`, Enterprise wave P0), which is designed but not built — it needs a container runtime this host does not currently have running.

9. Iteration lifecycle & restarts

An iteration ends `committed`, `rolled_back`, `interrupted` or `error`. A failed run is **classified**, not just recorded: `core/failure.js` maps the error to a `kind`, a human explanation, a remedy, and the phase a restart should resume from.

Restarts replay: the phases before the resume point are skipped as already-done, and the implementer receives its own previous edits plus the exact failure text, so it fixes rather than re-derives.

A **watchdog** aborts an iteration that makes no progress for `ITERATION_MAX_MINUTES` (default 30), so a hung model call can never freeze the loop.

Part III — The intelligence

10. The 13 agents

Each is a specialist with its own system prompt, tool set and area of competence:

security · tests · performance · quality · frontend · services · workbench · resilience · compliance · docs · infra · refactor · ux

They share one **bounded tool loop** (`IMPLEMENTER_MAX_STEPS`, default 20) with tools for reading, searching, writing, and — importantly — `search_knowledge`, which queries the hybrid index so a change is grounded in the whole architecture rather than a blind grep.

Path access is gated by `pathGuard.js` and `DENY_GLOBS` / `SECRET_GLOBS`: an agent cannot read a `.env` or write outside the product surface.

11. The 13 managers

Managers do not write code. They watch events, compute a picture, and publish a **brief** with a status (`ok` / `acting` / `alert`) and a recommendation:

Director · Quality · Throughput · Risk · Insights · Implementation · Operations · Services · Context · Reliability · Compliance · Deployment · Workbench

The **Director** is the one that matters most to an operator: it names the single critical task right now, and the dashboard surfaces it at the top of Overview.

Analysis is **debounced** — a burst of events collapses into one pass — because a manager that recomputes per event turns a busy loop into a CPU fire.

12. The Decision Network

The fleet's learned routing brain. For every (agent, area) pair it records attempts, landings and failures, and derives a land rate. The planner blends this with a static file→specialist prior to route each task to whoever is most likely to *ship* it.

The dashboard renders it as a **competence heatmap** whose colour encoding is deliberately careful: hue carries the land rate, paleness carries how little evidence stands behind it, and a pair with fewer than three attempts is drawn as explicitly *not judged* — no percentage at all. One lucky attempt is 100% and would otherwise glow brighter than 35-of-50, which is far stronger evidence.

13. Shared Memory

Durable, scoped lessons every agent reads on its next run. Kinds: `lesson`, `pitfall`, `pattern`, `fact`. Scopes: `global`, `area:<name>`, `agent:<id>`.

Memory is written automatically from:

- **failures** — the classified cause becomes a scoped pitfall

- **human rejections** — the strongest signal ISL gets, written as an agent-scoped pitfall naming the risk and the reasons
- **landed changes** — indexed as proven approaches for similar future work
- **context and manager analysis** — durable facts about the project

14. Context — the project's ground truth

Before the agents can plan, ISL reads the repository and writes down what it is: languages, layout, services, entry points, hotspots, conventions. It ingests documentation (Markdown, PDF via `pdf-parse`, DOCX via `mammoth`) and can ask the operator **questions** when something is ambiguous.

Without this, the planner plans against a codebase it has never seen — which produces work that looks plausible and fits nothing.

15. The knowledge index (hybrid RAG)

`context/knowledgeIndex.js` builds a retrieval index over every code file (path + exported symbols + head), every shared-memory lesson, and every landed change. Two signals are blended:

- **lexical** — BM25 over a camelCase-aware tokenizer, with an exact-symbol-match boost
- **vector** — cosine similarity of `nomic-embed-text` embeddings, content-hash cached

Exposed as the `search_knowledge` tool to the implementer and to Alfred (the chat assistant), and as `similarChanges(query)` which retrieves the most similar **past landed change** so the implementer reuses a proven approach.

A design correction worth recording: backlog de-duplication was first built on sentence embeddings and was wrong. The backlog is full of templated titles ("Fix accessibility in Privacy.jsx" vs "...Insurance.jsx") that embed at ~0.95 cosine while targeting *different* files — 550 false pairs from 34 items. It now keys on same-file + same-intent and finds 5 precise pairs. Sentence similarity is the wrong tool for a templated corpus.

16. Models & LLM providers

Per-role model selection. `implement`, `review`, `security`, `plan` and `research` can each use a different model — a cheap one for triage, a strong one for the phases that decide quality. Precedence: operator setting → environment variable → built-in default.

Provider abstraction. Default is local Ollama. Setting `ISL_LLM_PROVIDER=openai` routes to any OpenAI-compatible chat-completions API.

Detection. `core/models.js` asks the configured host what it actually has, so Settings offers real choices rather than a typed string.

Every call — local or remote — passes the egress firewall (\$23) and the cost meter (\$27), because both are wired at the single choke point in `src/ollama.js` rather than at each call site.

Part IV — The guardrails

17. Deterministic gates

`securityGate.js` reads the unified diff and flags, without a model:

- **Secrets** in added lines — private keys, AWS/Stripe/GitHub/Google/Slack credentials, JWTs, hardcoded password assignments, plus an **entropy heuristic** for vendor-less tokens
- **Weakening changes** — `rejectUnauthorized: false`, `Math.random()` for security material, `eval`, shell interpolation, weak hashes, disabled security lint rules
- **Removed controls** — an auth/ownership check deleted from a `-` line

`safetyGate.js` covers destructive change: truncating a table, dropping a column, removing a migration.

Test and fixture files are exempted from the secret veto — dummy tokens there are legitimate.

17.1 Intent — did the change do what the task said?

Every gate above asks whether a diff is *broken*. None asks whether it is the change that was *requested*, and that turned out to be a distinct failure mode with real consequences.

`intentGate.js` implements two rules, both deterministic:

- **Intent preservation.** A task whose title commits to adding or fixing something — *add*, *implement*, *resolve*, *wire*, *fix* — must not produce a diff in which the identifier it names is deleted and never re-added. The check is scoped to the files the task declared, so a cleanup task in one file cannot be blamed on an additive task in another.
- **The review floor.** A review score below `review_floor` (KPI, default 70) is disqualifying on its own, regardless of the weighted total.

The second rule exists because of arithmetic. Review carries a weight of 0.2 and is the only grader that compares the change against what was asked; the other four measure mechanical properties of the result. A review of 0 alongside four near-perfect mechanical scores totals 79 against a rollback threshold of 60 — so the one judgement in the system could never stop anything by itself. Measured across this project's history, **31 of 115 commits carried a review below 70**, four of them a review of 0. The committed scores fall in two clusters, 0–55 and 75–100, with nothing between; the floor sits in that gap.

Intent is judged from the **title**, not the rationale. Classifying on both looked more thorough and was the hole one failure went through: a task titled "Resolve TODO ... regarding userPrefs" justified itself as "unclear if it's dead code or a missing feature — removing this ensures the route is clean", and a gate reading the rationale saw a subtractive word and stood down. A model writing its own justification can always supply one.

17.2 Schema — a field with no migration is a column that does not exist

`schemaGuard.js` vetoes a scalar field added to a `.prisma` model when the same change ships no migration. Relations are excluded (they have no column behind them), and a re-indented model is not read as a set of new fields — adding one long name realigns the whole block, so every line arrives as `-old` / `+new`.

This gate exists because the failure is invisible to every other one. The file parses; it is not JavaScript, so nothing vanishes from any export; no suite covers it; and the service **boots**, because Prisma validates a field against the real table only when a query runs. One such change reached production and surfaced weeks later as "business

users have disappeared" — every read of that table asking Postgres for a column that had never existed.

The pre-commit guard is only half of it; §32.1 covers the agent that checks the deployed state.

18. Code intelligence

MODULE	ANSWERS
<code>blastRadius.js</code>	What breaks if this file is wrong? (reverse-dependency graph, tests, routes, sensitivity)
<code>structuralScan.js</code>	Which files are god-files or over-complex?
<code>coverageScan.js</code>	Which critical files are untested — or thinly tested?
<code>impactRank.js</code>	One composite leverage score: reach × untested × routes × sensitivity × size
<code>healthIndex.js</code>	A single deterministic 0-100 codebase health score, tracked over time
<code>refactorPlan.js</code>	How would this god-file split? (dry-run preview)
<code>frontendAudit.js</code>	ISL's own a11y/i18n gates, applied to the <i>target</i> app
<code>deadCode.js</code>	Did this change add code nothing calls?
<code>backlogDedup.js</code>	Which backlog items are the same work?
<code>bisect.js</code>	Which commit actually broke this?
<code>flaky.js</code>	Which tests fail non-deterministically?
<code>changelog.js</code>	Human-readable release notes from landed work
<code>digest.js</code>	The daily "what happened" summary

`blastRadius` is the one the others lean on. It builds a reverse-dependency graph from the target repo's own imports — no LLM, no guessing — and is cached with a short TTL and invalidated on commit.

19. Coverage

Two layers, and the difference between them matters.

Static proxy (`coverageScan.js`) — a file counts as covered if *any* test file imports it. Needs no test run, is never wrong about zero, works in any language. Its blind spot: a file whose only test asserts that it imports cleanly looks covered.

Measured (`coverageRun.js`) — runs the target's own suite under its own coverage tooling and parses the machine-readable report. Adapters for vitest, jest, pytest, go and cargo-llvm-cov; three parsers (istanbul `json-summary`, **LCOV**, go coverage profile) cover far more ecosystems than any single runner.

Safety properties:

- The suite runs in a **sandbox worktree**, so a test that writes files or seeds a dev database cannot touch the operator's tree.
- Reports go to ISL's `.data`, **never** into the target repo.
- **Uncommitted changes are replayed into the sandbox**, because a HEAD-only run on a dirty checkout measures a codebase nobody has. The result records `dirty` and is always reported stale, since its commit no

longer identifies what was measured.

- Every failure path degrades to `available: false` **with a reason** — and runner detection is split from coverage-plugin detection, so the reason is actionable ("vitest is installed in backend but its coverage provider is not — install it with `npm install -D @vitest/coverage-v8@2.1.9`", pinned to the vitest actually present, because the provider is peer-locked to its runner).

The measurement corrects the proxy in **both** directions. On RentAll: `middleware/auth.js` reads "no tests" statically but is 27.1% covered transitively through route tests, while `store/auth.jsx` — critical and sensitive — is **0%: never executed**.

20. The public-contract gate

Every other veto catches what the **tests** can see. A refactor that renames an exported symbol, drops a route, or removes a database column passes a fully green suite and breaks consumers downstream. `blastRadius` names the callers *inside* the repo; nothing protected those outside it.

`contractDiff.js` snapshots the public surface — HTTP routes and their mount prefixes, exported symbols across JS/TS/Python/Go, Prisma models with field types **and optionality**, enums, GraphQL types — and diffs two commits.

Ambiguity always resolves to the higher severity, because a gate that under-reports converts "we didn't check" into "we checked and it was fine":

CLASSIFIED BREAKING	WHY
Route or mount removed	A client calling it now gets a 404; a removed mount relocates every route beneath it at once
Export removed	Importers break with a green suite
Column/table/enum value dropped	Irreversible migration
Optional → required	Writes that used to succeed are now rejected — no happy-path test sees it
New required column	Every existing writer fails. Additive only if optional.

Declared breaking changes are **surfaced, not blocked**. Breaking an API deliberately is legitimate; doing it silently is not — so the reason says "UNDECLARED" when it was not declared.

A base snapshot that comes back empty against a non-empty head is **refused**, not reported: "the entire codebase is new and nothing broke" is the most dangerous output a gate can produce.

21. Human review & trust

Every landed change is classified by **risk** (blast radius, sensitivity, size) against the acting agent's earned **trust**:

- low-risk work by a `proven` agent → auto-approved, recorded
- risky work, or any work by a `probation` agent → held **pending** for a human

Trust is earned from land history and **demoted by human rejections**: an agent rejected by a majority of humans drops to `probation` with zero latitude regardless of how well its work passed the machine gates.

Health-gated autonomy. If the Health Index falls more than N points below a recent peak, ISL enters `stabilise` mode and revokes *all* auto-land latitude until it recovers. The loop keeps running; it just gets cautious exactly when the trend is wrong.

22. Self-correction & reliability

- **Auto-bisect on a fresh regression.** If a run's tests fail on code the diff did not touch, ISL re-runs the suite at the commit the iteration *started* from. Failing there too means the breakage is pre-existing — the grader says so, and a background bisect finds the real culprit with a one-click revert. (The bisect itself was fixed after it named an innocent commit when *every* commit in the window failed; it now requires a genuine good→bad boundary and otherwise reports `culprit: null` with the reason.)
 - **Flaky detection.** Non-deterministic failures are tracked and retried rather than blamed on the change in front of them.
 - **Reliability manager.** Errors are grouped, explained in human terms, and — on approval — fixed.
 - **The supervisor.** `supervisor.mjs` raises the heap and restarts the server within seconds on any exit, including a clean memory-watchdog blink at `ISL_MEM_LIMIT_MB`. The loop auto-resumes because `loopDesired` is persisted.
-

Part V — Enterprise controls

23. The egress firewall

Every phase sends source code, diffs, memory and retrieved chunks to a model. With a cloud provider that content leaves the perimeter. `core/egress.js` is the one choke point every outbound payload passes through, wired into `ollama.chat()` — which every phase already funnels through, so a phase added tomorrow is covered by construction — and into the embedding path, the highest-volume egress in ISL.

It fails closed, and the default is closed. Modes are `local-only` / `approved-vendors` / `any`, defaulting to `local-only`, so an install that upgrades into this feature cannot start sending code somewhere new. "Local" is decided by the **resolved hostname**, not the provider's name — an Ollama on a shared GPU box is remote and treated as remote.

Redaction reuses the security gate's own rules. `SECRET_RULES` and `highEntropyLiterals` are imported, never copied: a drifted second copy of those patterns is a credential leaving the building. Local calls are ledgered but **not** redacted — degrading a prompt to a model on the operator's own machine costs answer quality and protects nothing already read from disk.

A tripwire sits under the redactor. Pattern matching only removes what it recognises, and a `.env` of bespoke internal tokens matches nothing. So the contents of forbidden files are fingerprinted, and the call is **denied** if any of those lines appear in a payload — including a value quoted without its key, which is how a secret usually reaches a prompt.

A hash-chained ledger records one row per call: model, destination, bytes, redaction count, and the **SHA-256 of the payload** — the hash, not the text, because storing payloads would make the ledger a second copy of the customer's source to protect.

24. Tamper-evident audit & evidence packs

`platform/hashChain.js` is a shared, table-agnostic append-only chain used by **both** the egress ledger and the platform audit trail. Two copies of "is this history intact?" would eventually disagree, and the drifted copy would be the one declaring a tampered log clean.

It proves **integrity, not secrecy**: someone who can write the database can still append, but cannot rewrite what is already there without verification failing at exactly that row.

Retention is part of the design. A chain that cannot be truncated grows without bound and makes any retention policy break verification forever. `sealChain` records the head hash before archiving, verification **resumes from the seal**, and a gap with no seal behind it is still reported as tampering. Sealing a chain that does not verify is refused outright — otherwise a checkpoint would launder a tampered history into a trusted one.

Rows written before chaining are reported as `unchainedLegacy`, never counted as verified.

Evidence packs (`core/evidencePack.js`) assemble, for one landed change: the originating task, the plan the agent committed to *before* editing, every gate phase with its score and **raw inputs**, the diff, the human decision and who made it, the policy version that actually held it, and the integrity state of both chains. The pack commits to itself with a SHA-256 digest, and renders to a **self-contained HTML file** — no external stylesheet, script or font — that an auditor can read without ISL, without the network, and without a build step.

25. RBAC & segregation of duties

Five roles, scoped per project, deliberately **not** hierarchical:

ROLE	CAN
<code>viewer</code>	Read the project and its code
<code>operator</code>	...plus run the loop, edit the backlog and the scope
<code>approver</code>	...plus decide reviews and promote
<code>owner</code>	...plus change governance and grant roles
<code>auditor</code>	Read the audit chain and evidence packs — and nothing else, including no source

`auditor` is why the model is not a ladder: an outside auditor should verify the controls without being granted access to the customer's code. Modelling roles as levels would have silently handed it to them.

Segregation of duties is separate from capabilities. `review.decide` says you may approve changes; it never says you may approve *this* one. Authorship is compared inside `decideReview` — not at the route, because that is the only place every path to a decision passes through — so no combination of role grants can be assembled into self-approval. A platform admin is `owner` everywhere, and SoD still refuses them.

The refusal **throws** rather than returning a soft failure (a self-approval that "didn't work" but returned normally reads as a bug and gets retried), and surfaces as **HTTP 409, not 403**: the identity *is* permitted to approve changes — just not this one. Rejection is a decision too, so an author cannot bury their own change either. Every refusal is audited and raised as a notification.

Outstanding: OIDC/SAML SSO and SCIM provisioning. Deliberately not stubbed — ISL is the client in an OIDC flow and the server for SCIM, and shipping either unverified against a real identity provider would be the unearned assurance the rest of this layer exists to prevent. The RBAC model is the seam they plug into: group→role mapping targets `project_roles`, SCIM deprovisioning targets `destroyUserSessions`.

26. Policy-as-code

An organisation could not express "changes under `payments/**` need two approvals" without editing ISL's source — and could not prove afterwards which rules were in force. `core/policy.js` adds a versioned rule document evaluated over facts ISL already computes: touched paths, risk, sensitivity, area, agent, agent trust, change size.

Three decisions carry the weight:

1. **Policy can only TIGHTEN, never loosen.** A rule can force review or bar promotion; nothing in a policy document can grant latitude the built-in logic withheld. This is stricter than "replace the hard-coded gates" on purpose: a gap in a config file must not become a gap in the guardrails.
2. **Every matching rule is evaluated and the most restrictive outcome wins** — not first-match. Rule order is therefore irrelevant, so an operator cannot weaken a policy by accident.
3. **Validation refuses what would silently never fire.** An unknown condition key (`pathz` for `paths`) is rejected rather than ignored — a typo'd condition looks enforced and is not.

Simulation replays a draft against the real queue and reports what would change, which rules matched how often, and — importantly — **which rules never matched**, because a rule matching nothing is usually a typo'd glob rather than a rule with nothing to do.

The matched rule ids and the policy version are persisted on the review item, so an evidence pack cites the policy that actually held a change. Ships **empty and disabled**.

27. Cost & capacity governance

Health-gated autonomy governs *quality*; nothing governed *spend*. `core/costMeter.js` meters every model call from the same choke point as the firewall.

The honesty decision that shapes the module: on a local Ollama the currency cost is approximately zero, so a dashboard reporting "€0.00 spent" would be true and useless — the real constraint is **tokens and time**. Three resources are metered unconditionally (calls, tokens, milliseconds) and money is derived **only where a rate is configured**. Unpriced calls are counted and surfaced: a local model *is* free, an unconfigured cloud model is *unknown*, and conflating the two is exactly how a budget silently never fires.

- **Failed calls are metered too** — they consumed tokens upstream, and a meter that counts only successes under-reports precisely when things go wrong.
- **Soft** warns and signals degradation to cheaper models; **hard** stops *new* iterations while letting in-flight ones finish, because abandoning a run mid-flight leaves a half-applied change.
- **Efficiency returns `null`, not zero, when the denominator is missing** — "we spent X and produced nothing measurable yet" is the honest answer.

Showback: tokens and cost per landed change, and per health point gained.

28. The self-evaluation harness

Models, prompts and thresholds were changed by judgement, with no way to answer "did that make ISL better?" — the land rate moves for a dozen unrelated reasons.

`core/evalHarness.js` freezes a set of **real past changes** and replays them through the deterministic gates under a candidate configuration.

Two limits are stated in every result:

1. **Only the deterministic gates are replayed.** They are pure functions of a diff and a config, so replay is exact and free. The LLM phases are not — replaying them needs real calls, costs real money, and would not be reproducible. A harness that scored them from stored text would be measuring its own fiction.
2. **Agreement is only as strong as the labels.** A verdict counts as labelled only where a human actually decided; `auto` is a machine decision, and treating it as ground truth would score the gates against themselves. With few labels the harness reports `underpowered: true` rather than a percentage computed from three samples.

What *is* reliable with no labels is **verdict stability**: replay every diff under baseline and candidate and count how many verdicts flip and in which direction. A flip toward permissive is the dangerous one. `safeToPromote` is true only when nothing loosened.

A design bug the acceptance test caught: the first version defaulted unknown agent trust to `probation`, which always holds — so the size thresholds never applied to the ~50 cases without a recorded trust, and the harness reported **0 flips for a config that clearly loosens**. A harness blind to the knob it exists to evaluate is worse than none. Cases with recorded trust now replay at that trust; cases without replay at **every** level.

29. Governance, compliance & best practices

- **Quality gates** — the organisation's bar: minimum score, highest auto-landable risk, require a test, require a green build, block on critical CVE.
 - **Protected paths** — globs the fleet may never touch autonomously, enforced before any model sees the task.
 - **Repo-wide secret scan** — credentials committed *before* ISL arrived, not just in the current diff.
 - **Licence inventory** — every dependency licence, flagged by policy for legal review.
 - **Best-practices knowledge base** — rules across every supported language (including ABAP), with a compliance agent and manager that audit against them.
 - **Webhooks** — HMAC-SHA256-signed outbound notifications, so ISL fits an existing ops stack rather than demanding a watched dashboard.
 - **Dependency & CVE management** — `npm audit` per project, plus **auto-remediation** that computes the minimal safe bump in an isolated worktree and reports which CVEs it closes. `--force` is never used, so no breaking major bumps. Because nothing is installed, the upgrade is **not test-verified** — the output is an evidence-backed patch proposal, labelled as such.
-

Part VI — Steering & operations

30. Scope — who decides what gets improved

`core/scope.js` is the human's steering wheel: nine themes (`ux` , `frontend` , `backend` , `architecture` , `security` , `performance` , `tests` , `docs` , `infra`) with focus weights, hard caps, free-text directives and exclusions.

This exists because of a measured problem: ISL's strongest skill is writing tests, and left alone it aimed that skill everywhere — producing a stream of test tasks and few user-visible improvements. Defaults now deliberately favour user-visible and structural work: **UX 20 · Frontend 20 · Backend 20 · Architecture 15 · Security 10 · Performance 5 · Tests 5 (capped at 25%) · Docs 3 · Infra 2**.

The scope reaches the loop, the agents, the managers and the online researcher, so "focus on UX this month" changes what is proposed, not just what is displayed.

31. Autonomy & scheduling

- **The loop** — interval-driven, cancellable, watchdogged; `loopDesired` is persisted so it resumes after a restart.
- **Improvement windows** — heavier passes (CVE scan, coverage/structural seeding, health snapshot, cross-project transfer, embedding build) run only during operator-defined quiet hours, each at most once per calendar day. Rendered as a 7 × 24 grid, because four numbers do not tell you when maintenance fires — and an **overnight window** (22 → 05) puts the early hours on the *previous* day's selection, which no sentence makes obvious.
- **Cross-project transfer** — a pattern learned on one project is offered to another.

32. Deployment, promotion & cloud

- **Promotion** — a fast-forward from the work branch to the base branch, refused on a non-fast-forward, a dirty tree, or the wrong checkout.
- **Trust-gated auto-promotion** — only a contiguous prefix of qualifying commits can move (walking base→tip and stopping at the first non-qualifying one), so it can never skip an unapproved change to ship a later one. Requires human approval or a `proven` agent, low risk, and autonomy not in stabilise mode. **Off by default**.
- **PR publisher** — opens a real pull request with the change, its evidence and its provenance.
- **Terraform drift** — infrastructure changes are checked against declared state.
- **Cloud** — GCP/AWS detection and a deploy manager.

32.1 The runtime, and what it may not do

`runtime/compose.js` drives the target application under docker compose. Two capabilities were added in 2026-08, both with an explicit safety boundary.

Per-container control. `stop` / `start` / `restart` / `rebuild` act on a single service. Before this the only controls were stack-wide, so the response to two crash-looping containers was to bring down the other six — Postgres included. `stop` uses compose's `stop`, not `down`: the container's volumes and network survive, so `start` returns the same instance. `rebuild` is separate and marked slow, because it re-runs the image build — which is the only way to re-run build-time steps such as `prisma generate`.

The flag that is refused. `docker compose down -v` removes named volumes, and the database lives in one. Losing it means losing every row, with no undo. Rather than trusting each future caller to remember, `assertNonDestructive` refuses any invocation carrying `-v`, `--volumes` or `--remove-orphans`, and `serviceArg` validates the service name so it cannot itself be a flag. A test in `test/composeSafety.test.js` reads the source and fails if a new command introduces one.

Adoption. The service list used to be populated only when ISL had started the stack itself, so a stack started from a terminal showed as "stopped" with an empty list — leaving every per-service control unreachable in exactly the situation an operator needs it. `controller.observe()` now reads `docker compose ps` and adopts what is running. It starts and stops nothing.

The schema integrity agent (`runtime/schemaAgent.js`) answers the question §17.2's guard cannot: not "does this diff introduce drift" but "is the deployed state consistent right now", whatever produced it — a migration that failed halfway, a schema edited by hand, a database restored from an older dump. It parses every `.prisma` model in the project and compares it against the live database's `information_schema`. It runs after every commit (non-blocking — a diagnostic must not fail the run it observes) and on demand from Runtime → Prisma schema.

It is **strictly read-only**: one `information_schema` SELECT, no DDL, no edit to any `.prisma` file. Drift is reported and never repaired. A tool that "fixed" this by altering either side would be able to destroy precisely what it exists to protect.

Schemas under directories named `legacy`, `archived`, `backup`, `old`, `deprecated` or `snapshot` are skipped: comparing an archived schema against today's database produces findings that are all correct and all meaningless, and a check made of noise is a check nobody reads.

33. Online research

The research agent searches the web for how comparable applications solve a problem, and turns findings into **complete proposals** — frontend and backend — rather than links. Scope-aware: it researches what the operator asked for.

Part VII — The dashboard

34. Navigation & shell

Six groups: **Workspace · Improve · Fleet · Deploy · Insight · Administration**. Every view is a lazy chunk, so the initial bundle is a small shell.

Persistent shell elements:

- **⌘K command palette** — jump to any view, run any action, and search the knowledge index inline
- **Top-bar status chips** — the model actually doing the work, and the current improvement focus (with an amber warning when the scope is advisory rather than enforced); both click through to their settings
- **Activity spotlight** — a live strip showing the running iteration, its phase across the ten-stage pipeline, and both clocks. Shown *only* while a run is in flight: a permanent "idle" bar would be furniture
- **Theme, density and locale** — persisted **per user, server-side**, with `localStorage` kept as the cache so there is no theme flash on load
- **Route error boundary** — a broken view does not take the shell down with it

35. The views

The nav carried 38 entries, several of which answered the same question from slightly different angles: following one run meant moving between Iterations, Runs and Flow. Five merges take it to 30.

VIEW	TABS	ANSWERS
☰ Overview		What is happening right now
🔦 Insight		Is ISL helping? — health, coverage, review backlog, cost efficiency
🗃 Projects · 📄 Context · 🗂 Explorer		The codebases, what ISL understands, the files
🎯 Scope		The improvement focus
🔄 Runs	Problems · All runs · Live	One run seen three ways: what failed, every iteration with its diff, what is happening now. Filterable by outcome and by text over 250 runs
💠 Plans · ☰ Backlog · 💠 Proposals · ✓ Summary		The committed plans, the work queue, the proposals, what landed
👤 Agents · 🛡 Managers		The fleet and its supervisors
☑ Review		The approval inbox (see below)
🔒 Security & Rules	Gate findings · Best practices · Policy	Three faces of "what is allowed": the diff gate, the rule catalogue, the enterprise policy
🔗 Reliability · 💬 Models		Failures and repairs; the local models and their assignment
📖 Memory	What it knows · How it routes	Learned lessons, and the agent × area competence graph

VIEW	TABS	ANSWERS
 Services		Integration health
 Workbench	Boot check · Diagnose & repair	Does the app run — as a gate inside a run, and on demand with a repair attempt
 Runtime	Containers · Prisma schema	Per-container control, and the schemas with their integrity verdict (\$32.1)
↑ Promote · ☁ Cloud		Promotion and the cloud surface
 Digest		What the fleet actually improved, in plain language
 Metrics	Codebase health · Run analytics · Cost & tokens	Three pages of read-only measurement. KPI stays separate: it is configuration, not a report
⊙ KPI · ≡ Logs · 🔔 Notifications · ⚙ Settings		Weights and thresholds, the log tail, the feed, the loop's settings
🔑 Admin		Users, platform, dashboard configuration (admin only)

No route was broken by the merge. `dashboard/src/merged.js` maps every absorbed id — `#/flow`, `#/telemetry`, `#/compliance` and the rest — to its page and tab, so bookmarks, notification links and the `onNavigate('...')` calls scattered across views all still arrive. The command palette keeps each absorbed page as its own destination and matches on the **route id** as well as the label, so searching "telemetry" finds the tab now called *Cost & tokens*. Badge counts are rolled up onto the hosting page, or a count keyed to an absorbed id would have had nowhere to render and read as zero.

`components/TabbedView.jsx` renders the **existing view components unchanged** — a merge that rewrites view internals is a merge that drops features. Only the visible tab is mounted, because these views poll. Each panel carries its own error boundary and its own Suspense boundary: with the boundary above the shell, a crash or a cold chunk removed the tab bar along with the panel, dropping keyboard focus to `<body>`.

 ANALYTICS ·  TELEMETRY ·  KPI ·  LOGS ·  NOTIFICATIONS	MEASUREMENT AND HISTORY
⚙ Settings · 🔑 Admin	Configuration; users and platform

36. The components worth knowing

DiffViewer — per-file diff with a **dependency-free** syntax tokenizer and inline blast-radius annotation: each file header carries its dependent count, and expanding it lists the callers that *must keep working* plus a warning when no test imports it. The tokenizer is written to be **incapable of losing text** — every branch consumes and re-emits its input verbatim — and is verified by 18 tests asserting character-identical output across escaped quotes, template literals, regex literals, unicode, and HTML-looking characters. A highlighter that drops a character is worse than none: the reviewer approves a change they did not actually see.

ReviewInbox — keyboard-driven approval (j/k move, a/r decide, ? help, suppressed while typing). Shows the diff, risk, trust, the reasons it was held, **which policy rule held it**, and **who originated it**. Changes you authored are marked `yours` with Approve/Reject disabled and the reason stated *before* you act — making someone discover a control by hitting a 409 is a poor way to explain a rule.

CompetenceHeatmap — see §12. Every Tailwind class is spelled out literally, because Tailwind compiles what it finds in the **source**: a constructed `bg-${hue}-${step}` would render the whole heatmap colourless while every value looked correct in the DOM.

DataTable — virtualised (only the visible slice is in the DOM), sortable, column-configurable with the choice persisted per table id, degrading to stacked cards below `sm`. Applied where the shape is *actually* tabular: backlog and memory rows carry a title, description and hover actions, so collapsing them into 28px rows would lose information and they stayed rich lists.

ScheduleGrid — see §31. Membership is recomputed client-side with the same rule the server uses, so what is painted is what will fire.

OnboardingTour — **checks reality instead of tracking progress**: each step asks the API whether it is already done, so someone who set everything up before ever seeing it gets a completed checklist, not a tutorial for work they finished. Shows only when a non-optional step is missing; dismissal is persisted server-side.

Notifications — a durable feed with severity/kind filters and an action link on every item. The governance controls write to it through `notifyOnce`, which dedupes within a time window: a misconfigured provider fails on every phase of every iteration, and a feed buried under ten thousand identical rows has lost the property that makes it worth having. Every occurrence is still in the ledger and the audit chain; the dedupe governs only how often a *human* is interrupted.

Part VIII — Reference

37. Running ISL

```
npm install
```

```
npm run serve
```

```
npm start
```

```
npm --prefix dashboard run build
```

```
npm --prefix dashboard test
```

`npm run serve` launches the supervisor (raises the heap, auto-restarts on any exit). `npm start` runs the server directly with no auto-restart. Default port **7878**. The first login with the seeded admin email claims the account and sets its password.

Always launch the supervisor, not `src/server.js` directly. It restarts within seconds on any exit — crash, OOM, or the clean memory-watchdog blink at `ISL_MEM_LIMIT_MB` (default 1400). This fixed the "server goes down after a few hours" problem. The loop auto-resumes because `loopDesired` is persisted.

38. Configuration

Environment (`.env`), all optional:

VARIABLE	DEFAULT	MEANING
<code>PORT</code>	<code>7878</code>	HTTP port
<code>ADMIN_EMAIL</code>	—	Seed admin; first login claims it
<code>AUTH_SECRET</code>	dev value	Session secret — set this in production
<code>SESSION_TTL_HOURS</code>	<code>168</code>	Session lifetime
<code>DEFAULT_PROJECT_PATH</code>	RentAll path	Folder for the first seeded project
<code>OLLAMA_HOST</code>	<code>http://127.0.0.1:11434</code>	Local model host
<code>OLLAMA_MODEL</code>	<code>qwen3.6:latest</code>	Default model
<code>OLLAMA_EMBED_MODEL</code>	<code>nomic-embed-text</code>	Embeddings; empty disables vectors
<code>OLLAMA_NUM_CTX</code>	<code>32768</code>	Context window

VARIABLE	DEFAULT	MEANING
<code>ISL_MODEL_IMPLEMENT</code> / <code>_REVIEW</code> / <code>_SECURITY</code> / <code>_PLAN</code> / <code>_RESEARCH</code>	—	Per-role model override
<code>ISL_LLM_PROVIDER</code>	<code>ollama</code>	<code>openai</code> routes to an OpenAI-compatible API
<code>ISL_LLM_BASE_URL</code> / <code>ISL_LLM_API_KEY</code>	—	That API's endpoint and key
<code>ITERATION_INTERVAL_SECONDS</code>	<code>120</code>	Loop period
<code>ITERATION_MAX_MINUTES</code>	<code>30</code>	Watchdog: abort a stalled iteration
<code>MAX_ITERATIONS_PER_DAY</code>	<code>100000</code>	Opt-in throttle
<code>IMPLEMETER_MAX_STEPS</code>	<code>20</code>	Tool-loop budget per task
<code>WORK_BRANCH</code>	<code>agents/auto-improve</code>	Where changes land
<code>APPLY_MODE</code>	<code>branch</code>	<code>direct</code> applies to the working tree
<code>ISL_MEM_LIMIT_MB</code>	<code>1400</code>	Supervisor memory watchdog

Runtime settings — quality gates, protected paths, egress policy, budgets, policy document, scope, schedule, models — live in the database and are edited from the dashboard, not from env.

39. API reference

201 endpoints. By area:

AREA	COUNT	COVERS
<code>governance</code>	34	gates, protected paths, egress policy + ledger + seal + preview, audit + seal, evidence packs, RBAC, policy + simulate, cost + budget + rates, eval + freeze + compare, secrets, licences, webhooks
<code>runtime</code>	10	boot, stop, logs, status
<code>control</code>	9	loop start/stop/cancel, run once, restart
<code>admin</code>	9	users, dashboard config, platform settings
<code>backlog</code>	8	list, add, update, duplicates, dedup
<code>proposals</code>	6	list, approve, reject, reverify
<code>deploy</code>	6	plan, promote, drift
<code>runs</code> · <code>models</code> · <code>files</code> · <code>context</code>	5 each	
<code>regression</code> · <code>orchestrator</code> · <code>memory</code> · <code>iteration</code> · <code>dependencies</code> · <code>coverage</code>	4 each	

AREA	COUNT	COVERS
summary · scope · schedule · iterations · flaky · compliance · chat · bestpractices · auto-promote · agents	3 each	
structural · review-queue · preferences · contract	2 each	

Notable individual endpoints:

GET /api/contract	the public surface snapshot
GET /api/contract/diff?base=<ref>	what this change breaks
POST /api/coverage/measure	start a real coverage run
GET /api/coverage/measured	its state and result
GET /api/governance/evidence/:id	an evidence pack (?format=html for the auditor's copy)
POST /api/governance/policy/simulate	what a draft policy would have changed
POST /api/governance/eval/compare	is this configuration safe to promote?
GET /api/governance/egress/ledger	what left, and to whom
GET /api/preferences	per-user settings (scoped by session, never by a body param)

40. Data model

43 tables. Platform (platform.db):

projects · users · sessions · audit · audit_checkpoints · platform_settings · egress_ledger ·
egress_checkpoints · model_usage · user_prefs · project_roles

Per project (agents.db):

agents · proposals · runs · events · messages · settings · kpi · logs · iterations · phases · tasks ·
plans · features · functions · notifications · golden_surface · health_snapshots ·
improvement_signals · flaky_events · error_events · anomalies · memory · documents · doc_findings ·
context_kv · context_questions · knowledge_embeddings · manager_briefs · manager_messages ·
best_practices · compliance_runs · compliance_findings · deploy_plans · terraform_findings · webhooks
· review_queue

Two tables are **hash-chained** (audit , egress_ledger) with a checkpoint table each. Schema migrations are additive ALTER TABLE ... ADD COLUMN guarded by try/catch, so an existing database upgrades in place.

41. Events

One WebSocket, one event stream. Types include hello , state , log , agent.token ,
agent.started/finished , impl.token , iteration.started/phase/finished/plan/replayed , proposal.* ,
verify.* , manager.brief , project.activated , config.changed .

An architectural rule this stream taught: the client's event log is a bounded tail (MAX_EVENTS = 500), and one chatty type — manager.brief at ~170/minute — fills it in under three minutes. A run lasts longer, so iteration.started is **evicted mid-run**. Any state derived by folding the log therefore concludes no run

exists and treats later phase events as orphaned. Run state is tracked incrementally in the store, where each event is seen once and nothing is evicted. **Never derive live state from a buffer that forgets.**

42. Source map

```
src/
  server.js          HTTP + WebSocket entry point
  config.js          live bindings, project resolution, guardrail globs
  db.js              per-project SQLite Proxy
  db_iteration.js    iteration/backlog/notification tables
  ollama.js          THE model choke point (egress firewall + cost meter)
  openaiProvider.js  OpenAI-compatible adapter
  bus.js logger.js glob.js files.js languages.js primitives
  orchestrator.js apply.js chat.js summary.js doctor.js tools/index.js

core/
  controller · scheduler · autonomy · scope · policy · costMeter
  egress · evalHarness · evidencePack · reviewQueue · trust
  decisionNetwork · crossProject · improvementWindows · governance
  models · failure · semaphore

iteration/
  engine · planner · parallelImplementer · implementer · graders
  securityGate · safetyGate · deadCode · contractDiff · changeBudget
  blastRadius · impactRank · structuralScan · coverageScan · coverageRun
  healthIndex · refactorPlan · frontendAudit · backlogDedup · bisect
  flaky · changelog · digest · regression · researcher · websearch
  langRunners (53 adapters) · sandboxTools · pathGuard · cataloger
  surveyor · promote · llm

platform/
  platformDb · hashChain · rbac · projects · activeProject · users
  authMiddleware · routes · featureRoutes · routes/codeIntelRoutes
  routes/governanceRoutes · dbExplorer

context/
  contextManager · contextAgent · contextDb · codeScan · extract
  ingest · knowledgeIndex

agents/
  registry · runner

managers/
  index (13 managers) · baseManager

memory/
  memoryDb

deploy/
  deployManager · autoPromote · prPublisher · depScan · cveRemediate
  cloudDetect · deployDb

sandbox/
  worktree · verifier

workbench/
  workbench · workbenchAgent

reliability/
  reliabilityManager

bestpractices/
  bestPracticesDb · complianceDb · complianceManager · seed · seedExtended

runtime/
  controller · compose · opsAgent

services/
  inventory

dashboard/src/
  App.jsx store.js hooks.js api.js nav.js i18n.js
  views/          36 lazy-loaded views
  components/     26 components + 11 test files (72 tests)

supervisor.mjs      heap sizing + auto-restart
ISL.md              this document
ISL_IMPROVE.MD      the backend improvement roadmap and its implementation log
ISL_Frontend_improvment.MD  the dashboard roadmap and its implementation log
```

43. Troubleshooting

SYMPTOM	CAUSE AND FIX
Model calls refused	The egress policy is <code>local-only</code> and the provider is remote. 🚫 Governance → Data egress.
"budget stop" in the logs	A hard cost budget was reached. 🚫 Governance → Cost.
Approve returns 409	Segregation of duties: you authored that change. Another eligible reviewer must decide.
Coverage says "not measured"	Install the coverage provider it names — the command is pinned to your runner's version.
Contract diff refuses to run	The base commit yielded an empty surface; the comparison would be meaningless. Check the ref.
The loop stops after hours	You launched <code>src/server.js</code> directly. Use <code>npm run serve</code> .
Dashboard changes do not appear	Rebuild: <code>npm --prefix dashboard run build</code> .
A feature works on one project, not another	A live binding was captured into a module-level <code>const</code> . See §4.
An iteration is stuck	The watchdog aborts it after <code>ITERATION_MAX_MINUTES</code> ; ⚠️ Runs offers a restart from the right phase.

44. Glossary

TERM	MEANING
Iteration	One turn of the loop: plan → implement → grade → commit or roll back
Phase	One of the ten stages of an iteration
Veto	A deterministic refusal no score can override
Blast radius	What breaks if a given file is wrong
Land rate	Share of an agent's changes that survived every gate and committed
Trust	<code>probation</code> / <code>trusted</code> / <code>proven</code> — earned latitude to auto-land
Stabilise mode	Health is falling; all auto-land latitude is revoked
Golden surface	A snapshot of working behaviour, used to detect regressions
Work branch	Where ISL commits; <code>main</code> is never touched
Promotion	The separate, gated fast-forward from work branch to base
Scope	The human-set improvement focus
Egress	Anything ISL sends to a model
Evidence pack	A signed, self-contained record of one change's full change-control history
Segregation of duties	The identity that authored a change can never approve it

TERM	MEANING
Underpowered	A statistic with too few samples to conclude anything — reported as such, not as a number

ISL improves other software, and increasingly itself. Every claim in this document is either implemented in the source it names or explicitly marked as outstanding — that distinction is the point, and it is the same standard the system applies to its own measurements.